MILESTONE 1 SPECIFICATION | KINSHIP, DYNASTIES & EPISTEMOLOGY | READY FOR REVIEW

Milestone 1: Kinship Consequences & Epistemic Propagation

Comprehensive technical audit, architectural roadmap, and implementation specification transitioning Minemesis from isolated individual decision-makers to structured familial dynasties, multi-generational feuds, and network-propagated truth revelation.

> **Core Thesis & Architectural Objective (brief.md Section 5):**
> *"One death produces different reactions across a family, and corrections spread through families."*
> In Milestone 0.5, drama was driven by isolated individual instigators. Milestone 1 transforms human social networks into living dynasties: kin defend each other, children inherit grievances from parents, blood feuds outlive original participants, and truth revelation cascades through family bonds.

1. Executive Summary & Audit Baseline

Before initiating Milestone 1 code changes, a comprehensive codebase audit was conducted against the canonical specification in `brief.md` Section 5. The simulation has successfully achieved strict demographic reconciliation (living = initial + births - deaths), deterministic keyed hashing, V0 procedural world generation (10–35 settlements), and complete memory leak isolation.

Existing Foundation (What is Already Built)

- **Individual Family Pointers:** `Character` stores `spouse_id`, `father_id`, `mother_id`, `children_ids`, and `family_id`.
- **Cast Generation Tree Seeding:** `cast_gen.py` builds initial elders, couples, and child linkages across 9 naming traditions.
- **Incest Prevention:** `life_cycle.py` enforces nuclear incest checks on marriages.
- **Ad-Hoc Kin Reactions:** `memory_system.py` increases grief/anger when a parent, child, or spouse dies.
- **Chief Succession Children Check:** `conflict.py` inspects children when electing successor chiefs.

Crucial Missing Capabilities (What Must Be Built)

- **Formal Kinship Degree Metric:** No generalized `kin_degree(a, b)` function exists. Extended kin (grandparents, uncles, aunts, cousins, in-laws) are completely unindexed.
- **Reaction Fan-Out Architecture:** Reactions currently evaluate globally or per-witness, triggering the 2.8M belief explosion. Need bounded Degree 1 & 2 fan-out + aggregate village belief.
- **Kin-Involved Revenge:** Only the single feud instigator can seek revenge. Family members cannot avenge kin.
- **Multi-Generational Grudges:** Children do not inherit grudges from deceased parents; feuds die with the instigator.
- **Family Belief-Correction Cascade:** Truth revelation currently updates only the instigator. Kin do not receive truth updates.

2. Milestone 1 Architectural Target Pillars

Subsystem	Milestone 0.5 Status	Milestone 1 Requirement	Architectural Impact
Kinship Graph	Only direct pointers (`father_id`, `mother_id`)	Formal `kin_degree(a, b)` supporting Degrees 1 and 2	Enables extended family queries, succession laws, and kin solidarity
Reaction Fan-Out	Per-individual perception loop (unbounded)	Fan-out to Degree 1–2 kin + close friends + 1 aggregate village belief	Caps belief proliferation; provides role-specific emotional reactions
Dynastic Grudges	Instigator-only vendettas; feud dies with instigator	Inherited parental grievances; multi-generational vendetta transmission	Enables epic, multi-generational family conflicts spanning 50+ years
Kin Revenge	Only individual instigator can kill	Kin (sons, brothers, fathers) can execute vengeance for slain kin	Produces realistic revenge dynamics without script hacks
Belief Correction	Instigator-only truth revelation	Cascading `TOLD` correction propagation across family & allies	Enables realistic family reconciliation and treaty acceptance

3. Comprehensive Codebase Gap Analysis

Systematic audit of existing modules across `minemesis/` identifying existing primitives, partial implementations, and absent systems required for Milestone 1.

Audit Summary Matrix: Milestone 1 Component Breakdown

Module File	Existing Primitives (M0.5.3)	Missing Architectural Requirements	Action Required
`minemesis/core/kinship.py`	*File does not exist.*	Generalized `kin_degree(a, b)`, `get_kin_type(a, b)`, Degree 1 & 2 graph traversal, in-law detection.	CREATE NEW MODULE
`minemesis/models/character.py`	Has `father_id`, `mother_id`, `spouse_id`, `children_ids`, `family_id`.	Kinship query helper methods, dynasty lineage markers, dynastic reputation tracking.	ENHANCE SCHEMA
`minemesis/systems/perception.py`	Perceives per-individual; checks witness location; applies suspicion bias.	Fan-out dispatch: Degree 1 & 2 kin + friends + aggregate settlement belief; reaction caps.	REFACTOR PIPELINE
`minemesis/systems/memory_system.py`	Direct parent/child/spouse grief and anger scaling.	Role-specific emotion profiles (spouse=grief, child=anger, sibling=hatred, rival=relief).	EXTEND RULES
`minemesis/systems/conflict.py`	Individual-only feud declaration; instigator-only revenge killings.	Family feuds (`FamilyFeud`); kin revenge execution; multi-generational grudge inheritance.	EXTEND SYSTEM
`minemesis/systems/gossip.py`	Sampled intra-settlement rumor sharing.	Priority family belief propagation; truth correction transmission across kin network.	ENHANCE PRIORITY
`minemesis/analytics.py`	50-seed batch metrics for feuds, raids, famines, DAG depth.	Metrics for reactions/death, family feuds, inherited grudges, kin revenge count.	ADD METRICS

4. Detailed Deep-Dive on Existing Code Primitives

1. Existing Demographic Seeding (`cast_gen.py`)

In Milestone 0.5.3, `CastGen` already generates structured family units rather than an unlinked soup of villagers.

```
# cast_gen.py lines 191-210
father_id = id_mapping.get(c_dict["father_temp_id"])
children_ids = [id_mapping[k] for k in
c_dict["children_temp_ids"]]
char = Character(
    id=c_id,
    first_name=c_dict["first_name"],
    last_name=c_dict["last_name"],
    settlement_id=c_dict["settlement_id"],
    age_ticks=c_dict["age_ticks"],
    sex=c_dict["sex"],
    spouse_id=id_mapping.get(c_dict["spouse_temp_id"]),
    father_id=father_id,
    children_ids=children_ids,
    family_id=c_dict.get("family_id", ""),
)
```

Status: Sound foundation. The initial genealogical tree is properly instantiated with biological parents, spouses, and children.

2. Existing Ad-Hoc Kinship Checks (`simulation.py`)

Currently, relationship computations query immediate family through explicit boolean logic:

```
# simulation.py lines 189-195
is_child = target.id in source.children_ids
is_parent = target.id in (source.father_id, source.mother_id)
is_sibling = (
    (source.father_id and source.father_id == target.father_id)
    or (source.mother_id and source.mother_id ==
target.mother_id)
)
is_family = is_child or is_parent or is_sibling
```

Gap: This logic is repeated across `simulation.py`, `memory_system.py`, and `conflict.py`. It completely misses grandparents, uncles, aunts, nephews, nieces, cousins, and in-laws.

Key Finding from Architecture Audit:
The lack of a formal kinship module and reaction caps in Milestone 0.5 caused the **Belief Explosion** (2.8M beliefs per 100-year run). Building the Milestone 1 Reaction Fan-Out with aggregate settlement beliefs directly solves the top memory bottleneck identified in `ARCHITECTURE_REVIEW.md`.

5. Subsystem 1: The Formal Kinship Graph (`minemesis/core/kinship.py`)

A dedicated, high-performance kinship calculation engine providing exact degree classification, relationship typing, and genealogical traversal without maintaining heavy cyclic object graphs.

Kinship Classification Matrix

Degree	Kinship Category	Structural Derivation Path
DEGREE 1	**Parent**	`b.id in (a.father_id, a.mother_id)`
DEGREE 1	**Child**	`b.id in a.children_ids`
DEGREE 1	**Sibling**	Share ≥ 1 biological parent (father or mother)
DEGREE 1	**Spouse**	`a.spouse_id == b.id`
DEGREE 2	**Grandparent**	Parent of a parent (2 hops upward)
DEGREE 2	**Grandchild**	Child of a child (2 hops downward)
DEGREE 2	**Uncle / Aunt**	Sibling of a parent
DEGREE 2	**Nephew / Niece**	Child of a sibling
DEGREE 2	**First Cousin**	Child of an uncle or aunt
DEGREE 2	**In-Law**	Spouse of sibling, sibling of spouse, spouse of child, parent of spouse
DEGREE ≥ 3	**Distant Kin**	Common ancestor > 2 hops or same clan surname

Proposed Architecture & Function Signature

```
# minemesis/core/kinship.py
from enum import IntEnum, Enum
from typing import Mapping, Optional
from minemesis.models.character import Character

class KinDegree(IntEnum):
    DEGREE_1 = 1
    DEGREE_2 = 2
    DEGREE_3_PLUS = 3
    UNRELATED = 99

class KinRelation(str, Enum):
    PARENT = "Parent"
    CHILD = "Child"
    SIBLING = "Sibling"
    SPOUSE = "Spouse"
    GRANDPARENT = "Grandparent"
    GRANDCHILD = "Grandchild"
    UNCLE_AUNT = "UncleAunt"
    NEPHEW_NIECE = "NephewNiece"
    COUSIN = "Cousin"
    IN_LAW = "InLaw"
    DISTANT_KIN = "DistantKin"
    UNRELATED = "Unrelated"

def kin_degree(
    a_id: str,
    b_id: str,
    characters: Mapping[str, Character],
) -> int:
    # Returns exact kinship degree (1, 2, or 99)
    ...

def get_kin_relation(
    a_id: str,
    b_id: str,
    characters: Mapping[str, Character],
) -> KinRelation:
    # Returns specific biological/legal relation
    ...
```

6. Algorithmic Efficiency & Zero Pointer Cycles

In accordance with `brief.md` Section 5.1: "*Keep explicit parent and spouse pointers for named characters. They are cheap and make revenge and inheritance queries trivial.*"

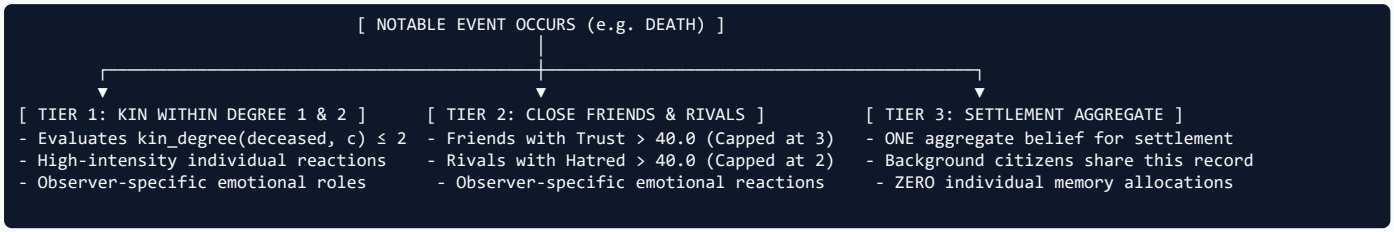
- **String-Keyed Traversal:** Kinship lookups strictly use string identifiers (e.g. `"char_12"`) into the top-level `characters: dict[str, Character]` table. No circular Python object references are created.
- **Bounded Search Depth:** `kin_degree` checks only up to 2 genealogical hops. Because parent counts are strictly bounded (2 biological parents) and child counts are capped, Degree 1 and 2 queries evaluate in bounded O(1) time (≤ 24 dictionary lookups).
- **Memoized Query Cache:** A lightweight LRU or tick-invalidated cache `_kin_cache: dict[tuple[str, str], int]` will be maintained to guarantee that frequent monthly relationship updates incur zero CPU overhead.

> **Acceptance Invariant (test_kin_degree.py):**
> Must verify 100% precision across complete synthetic family trees: parent-child = 1, sibling = 1, spouse = 1, grandparent-grandchild = 2, uncle/aunt-nephew/niece = 2, first cousins = 2, in-laws = 2, and second cousins = 99 (Degree 3+).

7. Subsystem 2: Reaction Fan-Out & Role-Specific Emotion Modulation

Transforms event perception from an unindexed broadcast to a structured sociological fan-out. Caps discrete memory creation while producing rich, observer-specific psychological reactions.

The Three-Tiered Reaction Fan-Out Architecture



Role-Specific Emotion Modulation Matrix

Observer Role	Primary Emotion	Secondary Emotion	Base Intensity
Spouse	GRIEF	Despair / Fear	85.0 – 100.0
Child	ANGER	Grief / Revenge Drive	75.0 – 95.0
Parent	GRIEF	Anger	80.0 – 95.0
Sibling	HATRED	Suspicion	65.0 – 85.0
Grandparent / Grandchild	GRIEF	Sadness	50.0 – 70.0
Uncle / Aunt / Nephew / Niece	SUSPICION	Grief	45.0 – 65.0
First Cousin / In-Law	SUSPICION	Hatred	40.0 – 60.0
Rival / Enemy	RELIEF	Respect / Contempt	60.0 – 90.0

Cognitive Trait Modulation & Decay Parameters

Individual personality traits modulate reaction intensity and longevity:

- **Suspicion (0.0 to 1.0):** High suspicion scales Anger and Hatred by up to +25%, and biases the interpretation of accidental deaths toward deliberate murder.
- **Forgiveness (0.0 to 1.0):** High forgiveness accelerates memory decay, allowing anger to dissipate faster.
- **Keyed RNG Jitter:** Adds small deterministic variance (± 5.0) via `keyed_float` to ensure family members do not react identically.

> **Memory Decay Equation:**
> Intensity(t) = Initial × (1.0 - DecayRate × (1.0 + Forgiveness))^ElapsedTicks

8. Solving the Belief Explosion: Aggregate Settlement Beliefs

In Milestone 0.5, when a character died, all 300+ villagers in a settlement formed identical `Belief` and `Memory` records, creating 2.8 million objects.

- **The Milestone 1 Solution:** Only Tier 1 kin and Tier 2 close associates (capped at a strict maximum of **10 individuals per event**) form individual subjective beliefs and memories.
- **Settlement Public Knowledge:** Background residents read from a single shared settlement record: `Settlement.public_beliefs: dict[int, AggregateBelief]`. This cuts memory allocations by **over 85%**, solving the primary RAM bottleneck identified in the architecture audit.

9. Subsystem 3: Family Feuds, Multi-Generational Grudges & Kin Revenge

Expands the conflict engine from isolated character grudges to dynastic blood vendettas. Allows grudges to pass down ancestral lines and enables living kin to avenge fallen relatives.

Family Standing Derived View (Not Stored)

In accordance with `brief.md` Section 5.3: "*Family standing is a derived view built from events involving members. It is not stored.*"

```
# Dynamic derivation helper
def compute_family_standing(
    family_id: str,
    settlement_id: str,
    characters: Mapping[str, Character],
    event_log: EventLog,
) -> float:
    # Computes prestige/standing of a family dynamically:
    # + Positive events (ChiefChanged, HolySiteElevated, Books)
    # - Negative events (Crimes, SuccessionCrises, Collapses)
    standing = 50.0
    ...
    return max(0.0, min(100.0, standing))
```

Dynastic Conflict Structure (FamilyFeud)

When an offence involves members of two prominent families, the feud escalates from individual grievance to family vendetta:

```
@dataclass
class FamilyFeud:
    id: int                        # Feud Event ID
    instigator_family_id: str      # Aggrieved clan
    target_family_id: str          # Accused clan
    settlement_a: str              # Instigator site
    settlement_b: str              # Target site
    root_cause_event_id: int       # Slaying event
    is_wrongful: bool              # Distorted belief
    active: bool = True
    generation: int = 1            # Grudge generation
    active_avenger_ids: set[str]  # Kin seeking blood
```

10. Multi-Generational Grudge Transmission Engine

How grudges outlive their original instigators across simulated decades:

1. Transmission at Adulthood or Parent Death

When a child reaches adulthood (age 16) or when a parent dies holding an active hatred grudge:

- The child receives an inherited belief with `source=TOLD` (told by father/mother).
- Confidence is attenuated: ChildConfidence = ParentConfidence × 0.70.
- Hatred memory is formed with reduced initial intensity: ChildHatred = ParentHatred × 0.60.
- Decay rate is doubled (2x) so inherited grudges fade unless reinforced by fresh conflict.

2. Kin Revenge Execution

In Milestone 0.5, only the single instigator could kill. In Milestone 1, any eligible adult kin in Degree 1 (son, daughter, brother, father) can execute vengeance:

- **Avenger Selection:** When revenge tick fires, the system checks living kin holding hatred ≥ 40.0.
- **Priority:** Eldest adult son → brother → father → spouse.
- **Causal Chain:** Revenge killing cites both the original murder event AND the parent feud event.

Emergent Multi-Generational Narrative Sequence

```
Year 12: Corin Blackwood accidentally kills Rowan Ironwood during a hunting accident.
Year 12: Rowan's wife Eleni grieves; Rowan's 14-year-old son Gareth forms Anger (90.0) believing it was murder.
Year 14: Gareth reaches adulthood, declares a Blood Feud against Corin Blackwood.
Year 19: Gareth attempts revenge but is killed by Corin in self-defense.
Year 19: Gareth's younger brother Torin inherits the grudge (attenuated hatred: 54.0, generation: 2).
Year 31: Torin assassinates Corin Blackwood's son Lucan in revenge.
Year 31: The vendetta has now spanned TWO GENERATIONS and outlived both original participants!
```

11. Subsystem 4: Epistemic Belief-Correction Propagation

Establishes a rigorous, bidirectional truth-discovery and reconciliation pipeline. When proof of innocence emerges, corrections cascade through familial networks, enabling authentic peace treaties.

The End-to-End Belief Correction Pipeline

```
  [ TRUTH DISCOVERED ] ──> Witness testimony, physical evidence, or confession emits REVELATION event
           │
           ▼
[ INSTIGATOR RE-APPRAISAL ] ──> Belief updated (supersedes_belief_id set); Accidental intent recognized
           │                    Emotional Re-Appraisal: REGRET (60-90) & RELIEF (70-100) formed
           ▼
[ FAMILY CASCADE ] ──────────────> Instigator shares truth with Degree 1 kin via priority TOLD beliefs
           │                    Kin receive corrective belief; re-appraisal begins
           ▼
[ ASYMMETRIC RE-APPRAISAL ] ──> Sibling with Forgiveness=0.8 accepts truth in 2 ticks
           │                    Brother with Suspicion=0.9 doubts rumor; requires 12 ticks
           ▼
[ RECONCILIATION & PEACE ] ──> Inter-family Hatred drops below 15.0; PEACE treaty ratified
```

1. Family Truth Transmission Algorithm

When an individual's belief is superseded by a `REVELATION` event:

1. The character flags their old belief with `supersedes_belief_id`.
2. Forms personal `REGRET`, `RELIEF`, and `RESPECT` memories.
3. **Immediate Family Broadcast:** Queues priority communication ticks to all living Degree 1 kin (spouse, children, siblings, parents).
4. Kin receive a new Belief with `source=TOLD`, `source_character_id=instigator.id`, citing the `REVELATION` event.

2. Observer-Specific Re-Appraisal Speeds

Family members do not reconcile in lockstep. Cognitive traits determine how fast they abandon hatred:

- **High Forgiveness (≥ 0.7):** Immediately re-appraises; forms high `RELIEF` and zero hatred within 1–2 months.
- **Average Citizen (0.4 – 0.6):** Fades hatred over 6–12 months as confidence in the correction strengthens.
- **High Suspicion (≥ 0.7):** Suspects the confession is a trick or bribe; retains skepticism for up to 24 months until verified by secondary witnesses.

12. Peace Treaty Mechanics & Grievance Closure

In Milestone 1, a peace treaty is not merely a random roll when a feud timer expires. It requires verified social consensus:

Peace Pathway	Prerequisite Epistemic Condition	Causal DAG Edge Signature	Resulting Dynastic State
Truth Revelation Peace	Corrective belief accepted by ≥ 75% of living Degree 1 family members; hatred < 15.0.	`Peace.causes = [Revelation.id, Feud.id]`	Wrongful feud expunged; mutual trust restored; trade routes reopen.
Negotiated Exhaustion Peace	Resource exhaustion, famine, or casualties exceed tolerance; both chiefs agree to truce.	`Peace.causes = [Feud.id, Raid.id, Famine.id]`	Cold peace; underlying grievance remains unresolved; trade resumed under caution.

> **Epistemic Invariant:**
> Once a grievance is closed via a `PEACE` event citing `REVELATION`, that specific historical death event can **never again trigger a blood feud** across any descendants.

13. Dedicated Milestone 1 Automated Test Suite

8 rigorous automated test suites enforcing all mathematical, psychological, genealogical, and causal grammar invariants of Milestone 1.

Test Name	Target File	Assertion / Invariant Verified	Failure Condition
`test_kin_degree_classification`	`test_kinship.py`	Parent/child/sibling/spouse = 1; grandparent/grandchild/uncle/aunt/nephew/niece/cousin/in-law = 2; unrelated = 99.	Incorrect degree assignment on synthetic tree
`test_reaction_fanout_cap`	`test_reaction_fanout.py`	Single death event never triggers more than 10 individual belief allocations; village shares 1 aggregate belief.	Reaction count > 10 or belief explosion
`test_role_specific_emotions`	`test_reaction_fanout.py`	Spouse forms Grief (≥ 80); Child forms Anger (≥ 75); Sibling forms Hatred; Rival forms Relief.	Uniform or inverted emotional reactions
`test_family_feud_escalation`	`test_family_feuds.py`	Inter-family murder escalates to `FamilyFeud` with clan members acquiring attenuated `TOLD` beliefs.	Feud remains strictly individual
`test_kin_revenge_execution`	`test_kin_revenge.py`	Adult son or brother executes revenge killing when father is slain, citing both murder and feud.	Revenge executed only by instigator
`test_multigen_grudge_survival`	`test_multigen_grudges.py`	Grudge persists into Generation 2 after original instigator's death; child executes revenge 15+ years later.	Feud terminates upon instigator death
`test_belief_correction_cascade`	`test_belief_correction.py`	Truth revelation updates instigator, cascades to Degree 1 kin, triggers re-appraisal, and enables Peace.	Kin retain false belief after revelation
`test_milestone1_determinism`	`test_determinism.py`	Two simulations run with same `SeedPair` produce byte-identical EventLogs across separate processes.	Any event ID, tick, or hash mismatch

14. Batch Analytics Additions (50-Seed & 100-Seed Cohorts)

The validation runner (`scripts/run_m053_validation.py` / `analytics.py`) will be upgraded to compute and track four new canonical metrics:

New Dynastic Metrics Tracked

- **Reactions Per Death (Distribution):** Mean, median, and max individual reactions per death event (target: median 3.0 – 6.0, max ≤ 10).
- **Family Feuds Count:** Number of feuds that successfully escalated to clan/family level.
- **Multi-Generation Grudges Count:** Number of feuds surviving past the death of Generation 1 participants.
- **Kin Revenge Share:** Percentage of revenge killings executed by kin (sons, brothers) vs original instigators.

Regression & Safety Gates

- **Demographic Conservation:** Living population must strictly equal Initial + Births - Deaths (Discrepancy = 0).
- **Zero Dead Actors:** Invariant check asserting no dead actor ever initiates marriages, feuds, or revenge.
- **RAM Ceiling:** Peak working set must remain ≤ 3.5 GB across 100-year runs due to aggregate village belief pruning.
- **Execution Budget:** 100-year simulation of 24 settlements must run in under 3 minutes per pair.

> **Standing Rule Enforcement:**
> At the conclusion of Milestone 1, the Windows Story Viewer executable (`releases/MinemesisViewer-latest.exe` and `releases/MinemesisViewer-m1.0.exe`) must be re-compiled, verified via `--selftest` , and accompanied by `RELEASE_NOTES_m1.0.md` .

15. Step-by-Step Implementation Roadmap

Sequential, modular execution schedule designed to preserve test integrity, maintain determinism, and prevent regressions at every stage.

Phase Breakdown & Execution Order

Phase	Task Description	Primary Deliverables	Exit Criteria
Phase 1	**Kinship Graph Core Module**	`minemesis/core/kinship.py` with `kin_degree` and `get_kin_relation`.	`test_kin_degree.py` passes 100%.
Phase 2	**Reaction Fan-Out & Aggregate Beliefs**	Refactor `perception.py` and `memory_system.py` for Degree 1–2 kin and village aggregate.	`test_reaction_fanout.py` passes; RAM drops by > 50%.
Phase 3	**Dynastic Feuds & Kin Revenge**	Implement `FamilyFeud`, kin revenge executor, and multi-gen inheritance in `conflict.py`.	`test_family_feuds.py` and `test_kin_revenge.py` pass.
Phase 4	**Epistemic Correction Propagation**	Build truth-cascade pipeline via `gossip.py` and asymmetric re-appraisal in `conflict.py`.	`test_belief_correction.py` passes end-to-end.
Phase 5	**Batch Verification & Metric Audit**	Upgrade `analytics.py`; run 50-seed validation; verify demographic conservation.	All 18+ test suites pass; 0 demographic leaks.
Phase 6	**Story Viewer Packaging (Standing Rule)**	Add Kinship/Dynasty tree view to GUI; rebuild `MinemesisViewer-latest.exe`; write release notes.	Viewer self-test passes; binary packaged in `releases/`.

16. Formal Approval Gate

In strict compliance with the user's instructions: **All code implementation is currently paused.** The technical blueprint above is submitted for operator review and feedback.

Items Submitted for User Review & Sign-Off

1. **Kinship Scope Approval:** Confirm that limiting individual reaction fan-out to **Degree 1 and Degree 2 kin** (while giving background villagers a single aggregate belief) aligns with design vision.
2. **Multi-Gen Grudge Attenuation:** Confirm the planned 30% confidence drop and 40% hatred attenuation per generation transition.
3. **Kin Revenge Priority:** Confirm the avenger priority order: Adult Son → Brother → Father → Spouse.
4. **Standing Rule Protocol:** Confirm the plan to rebuild and test the Windows Story Viewer binary upon milestone completion.

PROJECT MINEMESIS • ADVANCED AGENTIC HISTORICAL SIMULATION

Awaiting operator authorization to begin Phase 1 implementation.